MALLA REDDY UNIVERSITY

A Major Project Phase-2 Project Report

AIR-GAPPED USB FILE SANITIZER & MALWARE INSPECTION KIOSK

A Standalone IoT-Enabled Hardware Sandbox for Removable Media Defense

Design & Developed by

A. Sai Kumar	2311CS040001
A. Vaishnavi	2311CS040009
A. Narasimha	2311CS040011
A. Nikhitha	2311CS040013
A. Vaishnavi	2311CS040014
A. Abhiram	2311CS040017
G. Shivakrishna	2311CS040057
G. Manohar	2311CS040058
K. Sanjana	2311CS040083
K. Hemanth	2311CS040086

GUIDED BY

[Guide / Reviewer Name]

(Assistant Professor)

Department of Computer Science & Engineering (Cyber Security)

Malla Reddy University, Hyderabad

November-2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering (Cyber Security)

CERTIFICATE

This is to certify that this is the Bonafide record of the Major Project Phase-1 project entitled **"Air-Gapped USB File Sanitizer & Malware Inspection Kiosk"** submitted by **A. Sai Kumar (2311CS040001), A. Vaishnavi (2311CS040009), A. Narasimha (2311CS040011), A. Nikhitha (2311CS040013), A. Vaishnavi (2311CS040014), A. Abhiram (2311CS040017), G. Shivakrishna (2311CS040057), G. Manohar (2311CS040058), K. Sanjana (2311CS040083), K. Hemanth (2311CS040086)** of B. Tech IV year I semester, Department of CSE (Cyber Security) during the academic year 2026-27. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide
[Guide / Reviewer Name]
Assistant Professor

Dean (Cyber Security)
Dr. G. Anand Kumar
Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report entitled **"Air-Gapped USB File Sanitizer & Malware Inspection Kiosk"** has been carried out by us and submitted to the Department of Computer Science and Engineering (Cyber Security), Malla Reddy University, Hyderabad. This project work has been designed, developed & deployed by us, as part of the Project Work of this semester. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place:

Date:

A. Sai Kumar	2311CS040001
A. Vaishnavi	2311CS040009
A. Narasimha	2311CS040011
A. Nikhitha	2311CS040013
A. Vaishnavi	2311CS040014
A. Abhiram	2311CS040017
G. Shivakrishna	2311CS040057
G. Manohar	2311CS040058
K. Sanjana	2311CS040083
K. Hemanth	2311CS040086

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to **Dr. V. S. K Reddy,** Vice Chancellor, for his visionary leadership and unwavering commitment to academic excellence.

We are also grateful to **Dr. G. Anand Kumar,** Dean (Cyber Security), for providing us with the necessary resources and facilities to carry out this project.

We would also like to express our deepest appreciation to our project guide [Guide / Reviewer Name], Assistant Professor, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to our **PRC – convenor, Dr. G. Latha,** for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We extend our gratitude to our **Project Coordinator, Mr. A. Ramesh Khanna,** for his valuable support in review process.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

A. Sai Kumar	2311CS040001
A. Vaishnavi	2311CS040009
A. Narasimha	2311CS040011
A. Nikhitha	2311CS040013
A. Vaishnavi	2311CS040014
A. Abhiram	2311CS040017
G. Shivakrishna	2311CS040057
G. Manohar	2311CS040058
K. Sanjana	2311CS040083
K. Hemanth	2311CS040086

ABSTRACT

Removable media such as USB drives remain one of the most common ways of moving files between computers, and also one of the easiest ways of delivering malware, ransomware and malicious executables into trusted systems. Traditional endpoint protection may not identify every newly created, modified or unknown threat, and plugging an untrusted drive directly into a trusted computer exposes both the machine and its network.

This project presents the Air-Gapped USB File Sanitizer & Malware Inspection Kiosk, a standalone IoT-enabled hardware sandbox that acts as a security checkpoint between an untrusted USB device and a trusted computer. The kiosk is built around a Raspberry Pi single-board computer running Linux and a Python application. The user inserts the unknown USB into the kiosk; the system isolates the media, extracts the files, performs malware scanning together with AI/ML-based classification, assigns each file a risk level and a decision (APPROVED or QUARANTINE), and records the outcome as a timestamped JSON scan report. Approved files are transferred only through a clean output path, while suspicious or malicious files remain isolated in quarantine. Results are presented on a web dashboard served by the kiosk, and a security report is delivered by e-mail.

In the demonstration prototype, a scan of 7 files classified 4 as safe (APPROVED, LOW risk) and 3 script/executable files as QUARANTINE (MEDIUM risk). The project integrates Cyber Security, AI/ML, IoT and Embedded Hardware into a single, low-cost checkpoint suitable for colleges, banks, hospitals, government offices, laboratories and industrial networks.

Keywords: Air-Gapped Security, USB Sanitization, Malware Detection, Removable Media, Sandbox, Raspberry Pi, IoT, Embedded Hardware, AI/ML File Classification, Quarantine, Audit Logging, Cyber Security.

INDEX

Contents **Page No.**

Chapter 1 Introduction 1

1.1 Problem Definition & Description 1
1.2 Objectives of the Project 1
1.3 Scope of the Project 2

Chapter 2 System Analysis 3

2.1 Existing System Analysis 3
2.1.1 Background & Literature Survey 3
2.1.2 Limitations of Existing System 3
2.2 Proposed System Analysis 4
2.2.1 Advantages of the Proposed System 4
2.3 Software & Hardware Requirements 4
2.3.1 Software Requirements 4
2.3.2 Hardware Requirements 5
2.4 Feasibility Study 5
2.4.1 Technical Feasibility 5
2.4.2 Robustness & Reliability 5
2.4.3 Economic Feasibility 6
2.5 Requirement Specification 6
2.5.1 Functional Requirements 6
2.5.2 Non-Functional Requirements 6

Chapter 3 Architectural Design 8

3.1 Modules Design 8
3.2 Methodology 8
3.2.1 USB Detection & Isolation Algorithm 9
3.2.2 File Scanning & Classification Algorithm 9
3.2.3 Sanitization, Quarantine & Transfer Algorithm 9
3.2.4 Reporting Algorithm 10
3.3 Architecture 10
3.3.1 Complete Architecture 10
3.3.2 Data Flow & Process Flow Diagram 11
3.3.3 Class Diagram 11
3.3.4 Use Case Diagram 12
3.3.5 Sequence Diagram 13
3.3.6 Deployment Diagram 12
3.4 Report Data Structure 13
3.5 Security Design Considerations 14

Chapter 4 Implementation & Testing 16

4.1 Coding Blocks 16
4.1.1 Main Coding Block: main.py 16
4.1.2 Functional Code Block 1: Scanner & Classifier 16
4.1.3 Functional Code Block 2: Reporting, E-mail & Dashboard 18
4.1.4 Code Execution Flow Chart 19
4.2 Test Case Study 19
4.3 Test Summary 20

Chapter 5 Results **21**
5.1 Resulting Images 22
5.2 Resulting Tables 23
5.3 Resulting Graphs 24
5.4 Results & Analysis 24
5.4.1 Time Complexity 24
5.4.2 Space Complexity 24
5.4.3 Results Summary 24
5.4.4 Discussion & Limitations 25
5.5 Comparison with Existing Approaches 26
Chapter 6 Conclusion & Future Scope **27**
6.1 Conclusion 27
6.2 Future Scope 27
6.3 Learning Outcomes 27
References **28**

List of Diagrams

Contents		Page No.
Figure 2.3.2	Hardware Prototype	5
Figure 3.2.1	Methodology	8
Figure 3.3.1	Complete Architecture	9
Figure 3.3.2	Data Flow Diagram	10
Figure 3.3.3	Class Diagram	10
Figure 3.3.4	Use Case Diagram	11
Figure 3.3.5	Sequence Diagram	11
Figure 3.3.6	Deployment Diagram	12
Figure 4.1.4	Code Execution Flow Chart	17

List of Tables

Contents		Page No.
Table 5.2.1	Per-file Results of the Demonstration Scan	21
Table 5.2.2	Summary of the Demonstration Scan	22

List of Graphs

Contents		Page No.
Graph 5.3.1	Files by Decision and Risk Level Distribution	22

List of Screenshots

Contents		Page No.
Screenshot 5.1.1	Kiosk Hardware	19
Screenshot 5.1.2	Removable Medium Detection	20
Screenshot 5.1.3	Application Start and Report Listing	20
Screenshot 5.1.4	E-mail Report and Dashboard	21

CHAPTER - 1
INTRODUCTION

USB flash drives and portable disks are used everywhere to carry documents, datasets and software between computers. Their convenience is also their weakness: an unknown USB may carry malware, ransomware, malicious executables, auto-run scripts or booby-trapped documents. Once such a drive is connected to a trusted computer, the threat can run on that machine and spread across the connected network.

This project builds a dedicated inspection checkpoint, a kiosk, that sits between the untrusted USB and the trusted computer. The guiding principle is: **inspect first, isolate threats, transfer only what is safe.** The kiosk reads the untrusted media in an isolated environment, scans every file, and only releases files that have been approved.

1.1 PROBLEM DEFINITION & DESCRIPTION

USB drives are widely used for transferring files, but an unknown USB may contain malware, ransomware, malicious executable files, or suspicious documents. Connecting an untrusted USB directly to a trusted computer can expose the system and connected network to cyber threats. Traditional endpoint protection may not identify every newly created, modified, or unknown threat, particularly script files (.bat, .exe and similar) that look harmless to a user.

A dedicated inspection checkpoint is therefore needed before removable media reaches trusted systems. It must work independently of the trusted computer, keep the inspected media isolated, give a clear decision for every file, and leave an auditable record of what was found.

1.2 OBJECTIVES OF THE PROJECT

The primary objective is to design and develop a standalone, low-cost, hardware-based USB inspection kiosk. Specific objectives are:

1. Detect and safely read unknown USB devices.
2. Isolate removable media from trusted systems.
3. Scan files for known and suspicious threats.
4. Use AI/ML to classify suspicious files.
5. Perform controlled sandbox analysis.
6. Sanitize files where safe processing is possible.
7. Quarantine or block malicious files.
8. Transfer only approved files to a clean USB.
9. Generate security reports and audit logs.
10. Integrate IoT, AI/ML, Cyber Security and Embedded Hardware.

1.3 SCOPE OF THE PROJECT

The system covers inspection of removable storage at a physical checkpoint: device detection, file extraction, scanning, risk classification, quarantine, clean transfer, dashboard display and report generation. It is intended for colleges and universities, banks and hospitals, government and corporate offices, research laboratories, industrial networks, and cyber security training and testing labs. Network-wide threat intelligence and multi-kiosk management are outside the current phase and are listed as future scope.

CHAPTER - 2
SYSTEM ANALYSIS

2.1 EXISTING SYSTEM ANALYSIS

1. **Direct connection with endpoint antivirus.** The USB is plugged straight into the user's computer and relies on the installed antivirus. Limitation: the threat is already on the trusted machine, and unknown or newly modified threats may be missed.
2. **Disabling auto-run / blocking USB ports.** Limitation: blocks legitimate use and does not inspect content.
3. **Software-only scanning stations.** Standard PCs running antivirus. Limitation: they are themselves networked, can be infected, and offer no hardware isolation or clean-output control.
4. **Commercial USB sanitization kiosks.** Limitation: expensive, proprietary and hard to customize for small institutions.
5. **Manual inspection by IT staff.** Limitation: slow, inconsistent and not scalable.

2.1.1 Background & Literature Survey

- **Removable media as an attack vector.** Studies of malware propagation show that USB devices are a persistent route for worms, ransomware droppers and auto-run payloads, including into isolated (air-gapped) networks.
- **Signature-based detection.** Hash and signature matching reliably catches known malware but cannot detect new or modified samples.
- **Static and dynamic analysis.** Static analysis inspects file structure, type and strings without running the file; dynamic analysis observes behaviour in a sandbox or virtual machine. Combining both improves coverage.
- **Machine learning for malware classification.** Research on feature-based classifiers (file type, size, entropy, extension, header features) shows that ML can flag unknown suspicious files, although false positives and dataset quality remain challenges.
- **Embedded and IoT security devices.** Low-cost single-board computers such as the Raspberry Pi have been used for dedicated security appliances, making hardware checkpoints affordable.

2.1.2 Limitations of Existing System

1. Threats reach the trusted computer before they are detected.
2. Unknown and modified malware may bypass signature-based tools.
3. No physical isolation between the untrusted media and trusted systems.
4. Scanning PCs are networked and can themselves be compromised.
5. Commercial kiosks are costly and closed.
6. Manual checks are slow and inconsistent.
7. Limited audit trail of what was scanned and what was decided.
8. No controlled clean-output path for approved files.

9. Little feedback to the user at the point of insertion.
10. Poor suitability for training and testing labs.

2.2 PROPOSED SYSTEM ANALYSIS

The proposed system is a standalone kiosk built on a Raspberry Pi with an input USB port for the untrusted device, an output USB port / storage for the clean device, a dashboard on a touchscreen or connected display, and LED/buzzer indicators. The inspection pipeline is: **Untrusted USB → USB Input / Isolation → File Extraction → Sandbox → Malware Scan → AI/ML Analysis → (Safe) Sanitize → Clean USB**, and **AI/ML Analysis → (Threat) Quarantine → Alert / Report.**

Each file receives a status, a decision (APPROVED or QUARANTINE) and a risk level (LOW, MEDIUM, HIGH). Results are stored as JSON scan reports, shown on a local web dashboard and sent as an e-mail report. The kiosk is designed to operate air-gapped from the main network during inspection; networking, where used, is only for delivering the finished report.

2.2.1 Advantages of the Proposed System

1. Prevents direct exposure of trusted computers.
2. Adds an isolated, dedicated security layer.
3. Combines signature, static and AI/ML-based detection.
4. Quarantines suspicious and malicious files.
5. Transfers only approved files through the clean output path.
6. Generates timestamped reports and audit logs.
7. Low cost, using commodity hardware and open-source software.
8. Gives immediate feedback through the dashboard and e-mail report.
9. Modular, so scanners and models can be upgraded independently.
10. Useful as a cyber security training and testing tool.

2.3 SOFTWARE & HARDWARE REQUIREMENTS

2.3.1 Software Requirements

- Operating System: Raspberry Pi OS (Linux)
- Language: Python 3 (virtual environment .venv)
- Web framework: Flask (local dashboard on port 5000)
- Reports: JSON scan reports, local database and audit logging
- Security: hashing, static/dynamic analysis, sandbox / VM, quarantine directory
- AI/ML: machine-learning based file classification
- Notification: e-mail delivery of scan reports with attachment
- Remote access / display: SSH or remote desktop from a laptop; Chromium browser

2.3.2 Hardware Requirements

- Raspberry Pi single-board computer (Raspberry Pi 5 / Mini PC in the full design)
- Touchscreen / monitor
- USB input port (untrusted media) and USB output port / external storage (clean output)
- SSD / portable external storage drive
- USB isolation / control hardware
- Buttons, LEDs and buzzer connected through GPIO jumper wiring
- Cooling fan and heatsink; power supply
- Custom kiosk enclosure (prototype mounted on a base board labelled "Air-Gapped USB File Sanitizer & Malware Inspection Kiosk")

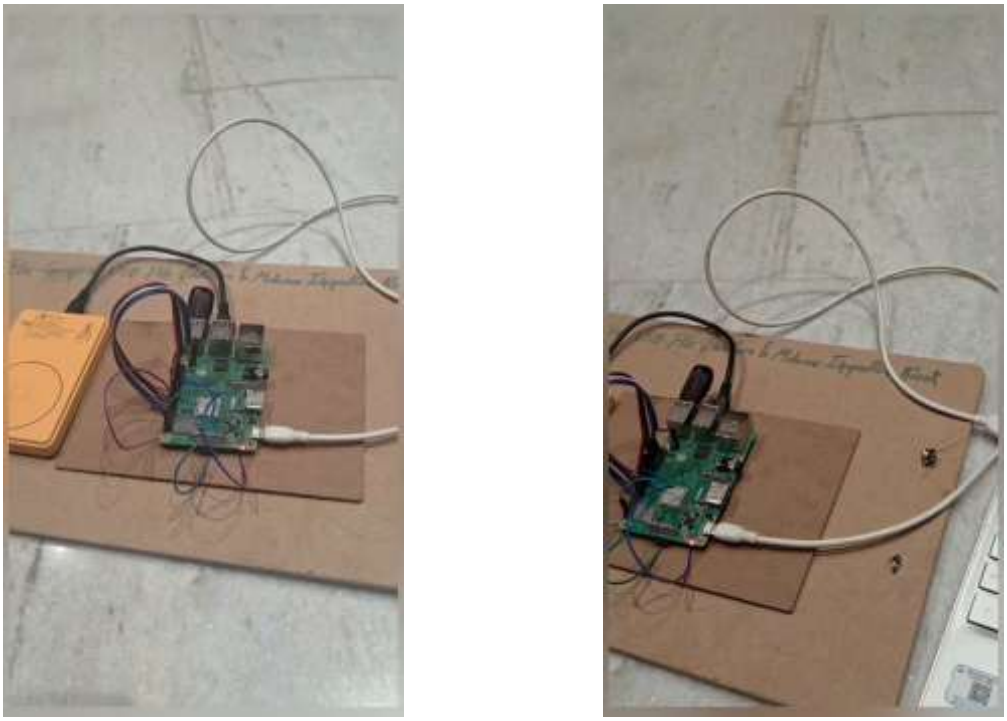


Figure 2.3.2: Hardware Prototype – Raspberry Pi kiosk with USB drive, portable storage and power cable

2.4 FEASIBILITY STUDY

2.4.1 Technical Feasibility

The project uses mature, well-documented technologies: Raspberry Pi hardware, Linux, Python and Flask. USB mounting, hashing, file-type analysis and GPIO control are all supported by standard Linux and Python libraries. The working prototype demonstrates that the full pipeline runs on a single-board computer.

2.4.2 Robustness & Reliability

Inspection runs on a dedicated device, so a compromised USB cannot touch the user's computer. Quarantine keeps threats isolated, every scan is saved as a timestamped report, and the stored report history gives a durable audit trail of all scans.

2.4.3 Economic Feasibility

The kiosk relies on an inexpensive single-board computer, commodity storage and open-source software, making it far cheaper than commercial sanitization stations. This makes it affordable for colleges, small offices and laboratories.

2.5 REQUIREMENT SPECIFICATION

The requirements below were derived from the problem definition and the objectives in Chapter 1. They describe what the kiosk must do and the qualities it must exhibit while doing so.

2.5.1 Functional Requirements

- **FR1 – USB detection:** the kiosk shall detect an unknown USB device inserted in the input port and mount it in an isolated location.
- **FR2 – File extraction:** the kiosk shall enumerate all files on the media and copy them to a controlled working area.
- **FR3 – Scanning:** the kiosk shall compute a hash and record the size, type and extension of every file and run malware and static checks.
- **FR4 – Classification:** the kiosk shall apply AI/ML based classification and assign a risk level of LOW, MEDIUM or HIGH to each file.
- **FR5 – Decision:** the kiosk shall assign a decision of APPROVED or QUARANTINE to each file.
- **FR6 – Quarantine:** the kiosk shall move suspicious or malicious files to the quarantine area and shall never transfer them.
- **FR7 – Clean transfer:** the kiosk shall copy only approved files to the clean output USB.
- **FR8 – Reporting:** the kiosk shall save a timestamped JSON scan report for every scan and append an entry to the audit log.
- **FR9 – Notification:** the kiosk shall e-mail the scan report as an attachment to the configured recipient.
- **FR10 – Dashboard:** the kiosk shall display the number of safe, suspicious and quarantined files and a per-file results table on a local web dashboard.
- **FR11 – Indication:** the kiosk shall signal scanning status and threat alerts through LEDs and a buzzer.

2.5.2 Non-Functional Requirements

- **Security:** the untrusted media must remain isolated from the trusted computer, and no file may be executed from the media during inspection.
- **Reliability:** every scan must produce a stored report so that results can be audited later.
- **Performance:** scan time should grow in proportion to the number and size of files, with immediate feedback to the user.

- **Usability:** the user should only need to insert the USB, read the dashboard and collect the clean USB.
- **Portability:** the system should run on common single-board computers and Linux mini PCs with open-source software.
- **Maintainability:** scanners and ML models should be replaceable independently of the other modules.
- **Cost:** the hardware and software should be inexpensive enough for colleges and small laboratories.
- **Auditability:** reports and logs must carry timestamps and a clear decision for each file.

CHAPTER - 3
ARCHITECTURAL DESIGN

3.1 MODULES DESIGN

1. **USB Detection & Isolation Module** – detects the inserted media and mounts it in an isolated location.
2. **File Extraction Module** – enumerates and copies files into a controlled working area.
3. **Sandbox / Analysis Module** – performs controlled static and dynamic analysis of files.
4. **Malware Scanning Module** – hash and signature checks and suspicious-content checks.
5. **AI/ML Classification Module** – classifies files using features such as extension, type, size and content indicators.
6. **Decision & Sanitization Module** – assigns status, decision and risk level; sanitizes approved files where possible.
7. **Quarantine Module** – isolates suspicious or malicious files.
8. **Clean Transfer Module** – copies only approved files to the clean USB / output path.
9. **Reporting & Notification Module** – writes JSON scan reports, sends the e-mail report and keeps audit logs.
10. **Web Dashboard Module** – Flask dashboard showing counts and per-file results.
11. **Hardware / IoT Control Module** – drives indicators and buttons through GPIO and monitors device state.

3.2 METHODOLOGY

The kiosk combines hashing, rule-based checks, static/dynamic analysis and ML classification to produce a risk-labelled decision for every file. Figure 3.2.1 shows the overall methodology: the untrusted USB is isolated, its files are extracted and scanned, threats are quarantined, and only approved files are sanitized and transferred.

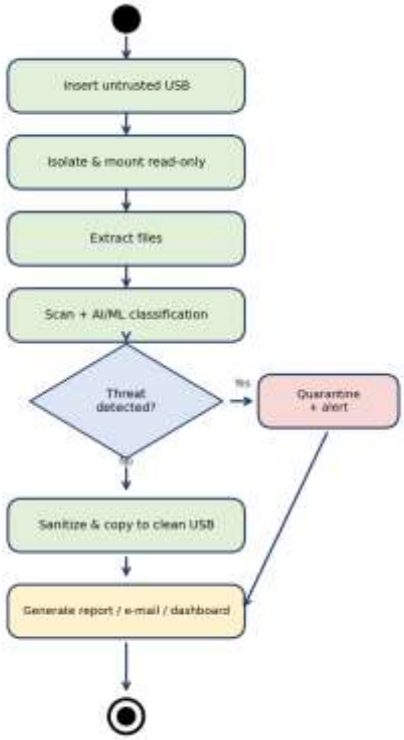


Figure 3.2.1: Methodology

3.2.1 USB Detection & Isolation Algorithm

```
Algorithm: USB_Isolation()
BEGIN
1. Wait for a USB device on the input port.
2. Identify the device and mount it in an isolated directory.
3. Indicate "Scanning" status.
4. Pass the mount point to File Extraction.
END
```

3.2.2 File Scanning & Classification Algorithm

```
Algorithm: Scan_And_Classify()
BEGIN
1. For each file on the media:
2.   Compute hash; record size, type and extension.
3.   Run malware and static checks (e.g. .exe, .bat scripts).
4.   Run sandbox analysis where required.
5.   Run AI/ML classification on extracted features.
6.   Assign risk level = LOW / MEDIUM / HIGH.
7.   IF safe: status = SAFE, decision = APPROVED.
8.   ELSE: decision = QUARANTINE.
9. Aggregate totals (safe, suspicious, quarantine).
END
```

3.2.3 Sanitization, Quarantine & Transfer Algorithm

```
Algorithm: Sanitize_And_Transfer()
BEGIN
1. FOR each APPROVED file: sanitize if possible, copy to clean USB.
2. FOR each QUARANTINE file: move to quarantine; do not transfer.
3. Update status indication and alert on threats.
END
```

3.2.4 Reporting Algorithm

```
Algorithm: Generate_Report()
BEGIN
1. Build report: file name, status, decision, risk level, time.
2. Save as scan_report_YYYYMMDD_HHMMSS.json in reports folder.
3. Update the web dashboard from the latest report.
4. E-mail the report as an attachment.
5. Append an entry to the audit log.
END
```

3.3 ARCHITECTURE

3.3.1 Complete Architecture

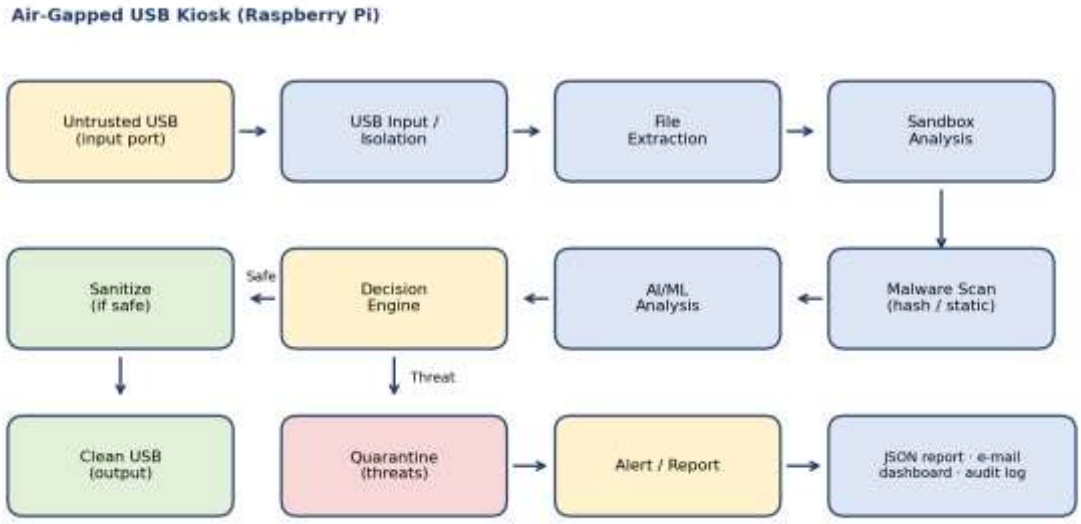


Figure 3.3.1: Complete Architecture

The untrusted USB enters through the isolated input port. Files are extracted and analysed inside the sandbox. Only files judged safe pass through sanitization to the clean output USB, while threats are quarantined and reported.

3.3.2 Data Flow & Process Flow Diagram

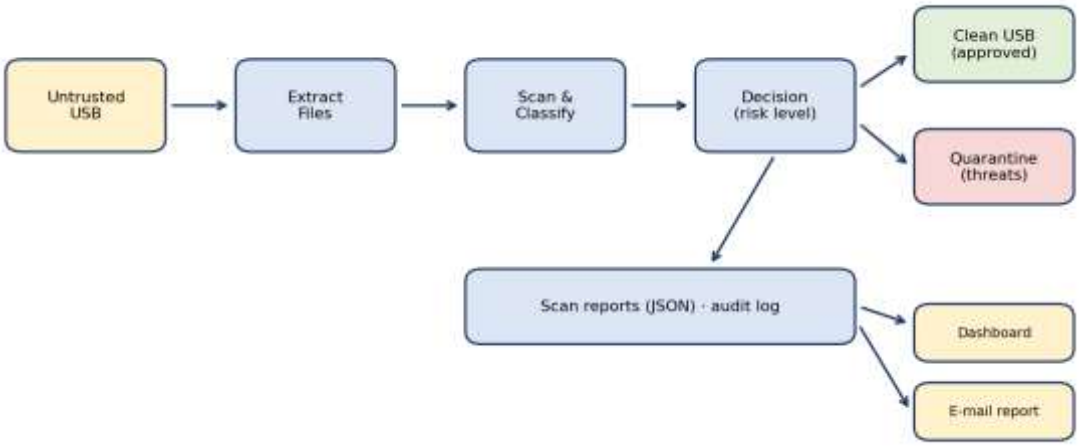


Figure 3.3.2: Data Flow Diagram

Data flows from the untrusted media to the extraction area, then to the scanner and classifier. Verdicts go to the decision stage, which routes approved files to the clean output and threats to quarantine. Every scan produces a JSON report that feeds the dashboard, the e-mail notification and the audit log.

3.3.3 Class Diagram

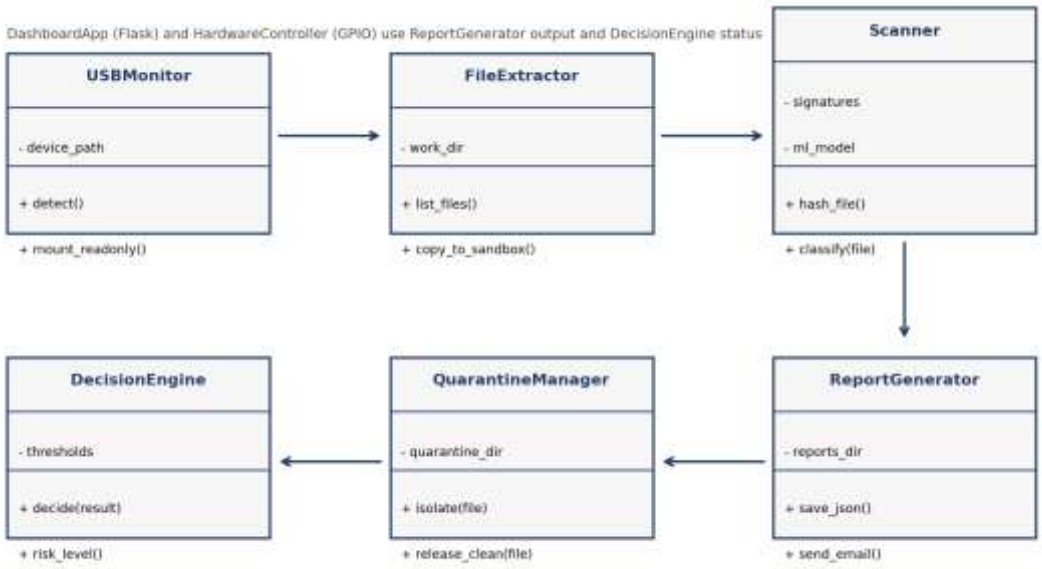


Figure 3.3.3: Class Diagram

The main components are USBMonitor, FileExtractor, Scanner (with the ML model), DecisionEngine, QuarantineManager and ReportGenerator, supported by the Flask DashboardApp and the hardware controller. Each class can be tested and upgraded independently.

3.3.4 Use Case Diagram

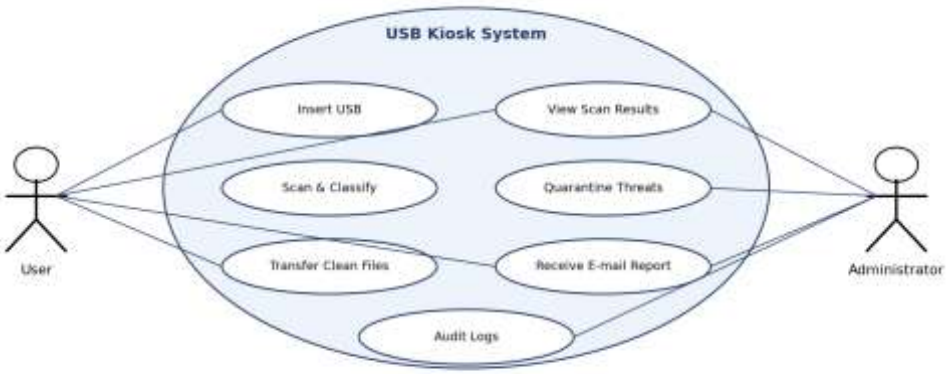


Figure 3.3.4: Use Case Diagram

The User inserts the USB, views scan results and collects the clean USB. The Administrator reviews results, quarantined files, e-mail reports and audit logs. The kiosk performs scanning, classification, quarantine and reporting.

3.3.5 Sequence Diagram

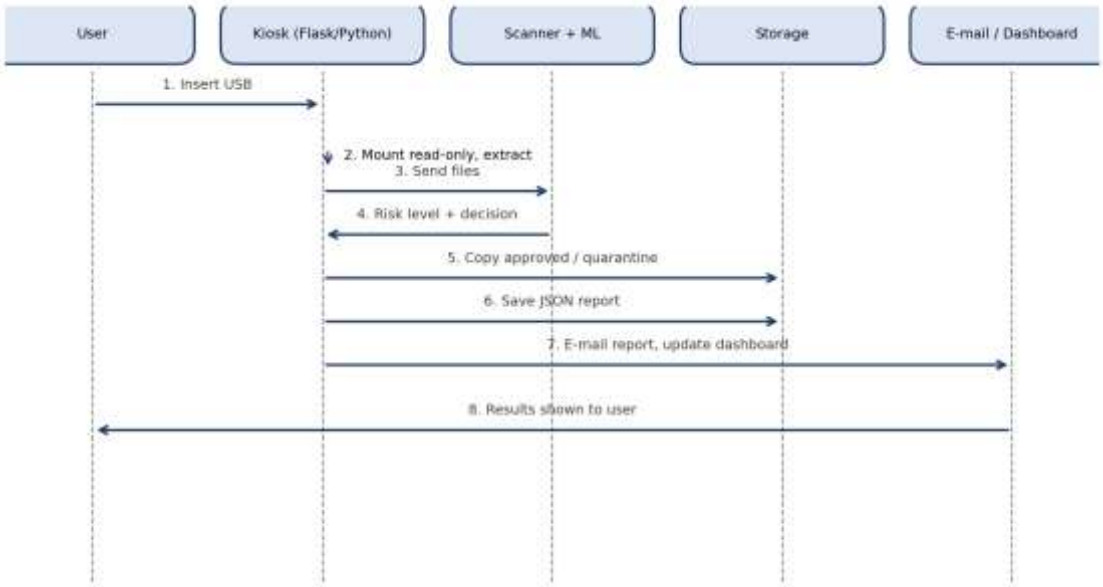


Figure 3.3.5: Sequence Diagram

The user inserts the USB; the kiosk mounts it and extracts the files, the scanner and ML model return risk levels and decisions, approved files are copied and threats quarantined, and the report is saved, e-mailed and displayed.

3.3.6 Deployment Diagram

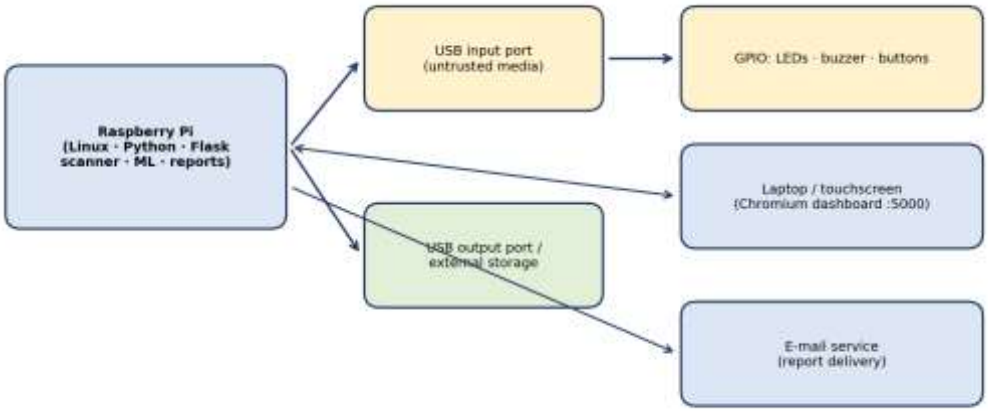


Figure 3.3.6: Deployment Diagram

The Raspberry Pi hosts the Linux OS, Python application, Flask dashboard, scanner, ML model and reports folder. It connects to the USB input port, the clean output storage, the GPIO components and the laptop/touchscreen browser, and to an e-mail service for report delivery.

3.4 REPORT DATA STRUCTURE

Each scan produces one JSON report named scan_report_YYYYMMDD_HHMMSS.json and stored in the reports folder. A report holds one record per file together with the scan time and the totals. Because the report is plain JSON, it can be read by the dashboard, attached to the e-mail and archived for audit without any conversion. Table 3.4.1 lists the logical fields used in each file record.

Field	Example	Purpose
File name	test.exe	Identifies the inspected file
Status	SAFE	Outcome of scanning and classification
Decision	QUARANTINE	APPROVED (transfer) or QUARANTINE (isolate)
Risk level	MEDIUM	LOW, MEDIUM or HIGH, from scanner and ML
Time	Scan timestamp	Records when the scan was performed

Table 3.4.1: Logical Fields of a Scan Report Record

The dashboard reads the latest report to display the summary cards for Safe Files, Suspicious and Quarantine and the per-file results table. Older reports remain in the folder, so the history of all scans forms the audit trail described in Section 2.4.2.

3.4.1 Decision Policy

The decision policy is deliberately conservative. A file is approved only when it is judged safe; every other file is quarantined. Table 3.4.2 summarises how the risk level relates to the action taken.

Risk level	Decision	Action taken by the kiosk
LOW	APPROVED	File is sanitized where possible and copied to the clean USB
MEDIUM	QUARANTINE	File is moved to quarantine and is not transferred
HIGH	QUARANTINE	File is moved to quarantine, an alert is raised and the incident is logged

Table 3.4.2: Risk Level and Decision Policy

3.5 SECURITY DESIGN CONSIDERATIONS

Because the kiosk handles untrusted input by design, the architecture is built on the assumption that any file on the media may be hostile. The following considerations guided the design of the modules.

- **Isolation first:** the media is mounted in a separate directory and is accessed only by the inspection modules, never by the trusted computer.
- **No execution of inspected files:** static checks examine files without running them; dynamic analysis is limited to the controlled sandbox.
- **Treat file names as data:** names and paths read from the media are handled as untrusted strings and are never passed to a shell unchecked.
- **Separate input and output paths:** approved files leave only through the clean output USB, so the input port can never act as an output path.
- **Fail safe:** when a file cannot be classified with confidence, it is quarantined rather than approved.
- **Minimal network exposure:** the kiosk is air-gapped during inspection, and networking is used only to deliver the finished report.

3.5.1 Threats Addressed

The design addresses the common ways in which a removable drive is used to attack a computer: auto-run scripts, malicious executables, batch and script files that run commands when opened, ransomware droppers, and documents carrying hidden content. Devices that impersonate a keyboard, as in the BadUSB work of Nohl and Lell, cannot be handled by file inspection alone and remain a limitation.

3.5.2 Trust Boundaries

Three trust zones are defined. The untrusted zone consists of the inserted USB and its files. The inspection zone is the kiosk itself, where files are copied, analysed and classified. The

trusted zone is the clean output USB and, beyond it, the user's computer. Files may cross from the untrusted zone into the inspection zone for analysis, but only approved and sanitized files may cross from the inspection zone into the trusted zone.

CHAPTER - 4
IMPLEMENTATION & TESTING

4.1 CODING BLOCKS

The application is written in Python and organised in an app folder (entry point main.py) with a reports folder for output, and runs inside a virtual environment on the Raspberry Pi. It is started with: source .venv/bin/activate, cd app, python main.py. On start it lists the stored reports, reads the latest one and launches the Flask dashboard.

4.1.1 Main Coding Block: main.py

```
import glob
import json
import os

from reporter import create_app

REPORT_DIR = os.path.join(os.path.dirname(__file__), "reports")


def list_reports():
    pattern = os.path.join(REPORT_DIR, "scan_report_*.json")
    return sorted(glob.glob(pattern))


def read_latest(reports):
    with open(reports[-1], encoding="utf-8") as f:
        return json.load(f)


if __name__ == "__main__":
    reports = list_reports()
    if not reports:
        print("No scan reports found in", REPORT_DIR)
    else:
        print("Stored scan reports:")
        for path in reports:
            print("  ", os.path.basename(path))
        latest = read_latest(reports)
        print("Latest report has", len(latest["files"]), "files")
    app = create_app(REPORT_DIR)
    app.run(host="0.0.0.0", port=5000, debug=False)
```

Description: main.py starts the kiosk, lists all stored scan reports (scan_report_<date>_<time>.json), reads the latest report, reports the number of files in it and launches the Flask dashboard.

4.1.2 Functional Code Block 1: Scanner & Classifier

```
import hashlib
import math
import os

RISKY_EXT = {".exe": "MEDIUM", ".bat": "MEDIUM", ".cmd": "MEDIUM",
             ".scr": "HIGH", ".vbs": "HIGH", ".ps1": "MEDIUM",
             ".js": "MEDIUM", ".jar": "MEDIUM", ".dll": "MEDIUM"}
EXE_MAGIC = (b"MZ", b"\x7fELF", b"#!")
```

```
def sha256_of(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(65536), b""):
            h.update(block)
    return h.hexdigest()


def entropy(data):
    if not data:
        return 0.0
    counts = [data.count(bytes([b])) for b in set(data)]
    return -sum(c / len(data) * math.log2(c / len(data)) for c in counts)


def extract_features(path):
    with open(path, "rb") as f:
        head = f.read(4096)
    return {"ext": os.path.splitext(path)[1].lower(),
            "size": os.path.getsize(path),
            "entropy": entropy(head),
            "exec_magic": head.startswith(EXE_MAGIC)}


def classify(features):
    """Feature-based risk scoring; a trained ML model can replace this."""
    score = 0
    if features["ext"] in RISKY_EXT:
        score += 2
    if features["exec_magic"]:
        score += 1
    if features["entropy"] > 7.5:
        score += 1
    if score >= 4:
        return "HIGH"
    return "MEDIUM" if score >= 2 else "LOW"


def scan file(path, known bad=frozenset()):
    digest = sha256_of(path)
    feats = extract_features(path)
    risk = "HIGH" if digest in known_bad else classify(feats)
    safe = risk == "LOW"
    return {"file": os.path.basename(path), "sha256": digest,
            "status": "SAFE" if safe else "QUARANTINED",
            "decision": "APPROVED" if safe else "QUARANTINE",
            "risk": risk}


def scan_directory(root, known_bad=frozenset()):
    results = []
    for folder, _dirs, names in os.walk(root):
        for name in sorted(names):
            results.append(scan file(os.path.join(folder, name), known bad))
    return results
```

Description: for each file the scanner records name, type and hash, applies malware and static checks, obtains the ML risk assessment and returns status, decision and risk level.

4.1.3 Functional Code Block 2: Reporting, E-mail & Dashboard

```
import json
import os
import smtplib
from datetime import datetime
from email.message import EmailMessage

from flask import Flask, render_template_string


def save report(results, report dir):
    os.makedirs(report_dir, exist_ok=True)
    stamp = datetime.now().strftime("%Y%m%d %H%M%S")
    path = os.path.join(report_dir, f"scan_report_{stamp}.json")
    data = {"time": stamp, "files": results}
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    return path


def send_email(path, host, user, password, to):
    msg = EmailMessage()
    msg["Subject"] = "USB Security Scan Report Generated"
    msg["From"], msg["To"] = user, to
    msg.set content(f"Report: {path}\nTime: {datetime.now()}")
    with open(path, "rb") as f:
        msg.add_attachment(f.read(), maintype="application",
                           subtype="json", filename=os.path.basename(path))
    with smtplib.SMTP_SSL(host, 465) as server:
        server.login(user, password)
        server.send_message(msg)


PAGE = """<h2>Air-Gapped USB File Sanitizer</h2>
<p>Safe Files: {{ safe }} | Suspicious: {{ susp }} | Quarantine: {{ quar }}</p>
<table border=1><tr><th>File</th><th>Decision</th><th>Risk</th></tr>
{% for r in files %}<tr><td>{{ r.file }}</td><td>{{ r.decision }}</td>
<td>{{ r.risk }}</td></tr>{% endfor %}</table>"""


def create_app(report_dir):
    app = Flask(  name  )

    @app.route("/")
    def dashboard():
        names = sorted(n for n in os.listdir(report_dir)
                       if n.startswith("scan report "))
        files = []
        if names:
            with open(os.path.join(report_dir, names[-1]), encoding="utf-8") as f:
                files = json.load(f)["files"]
        safe = sum(r["status"] == "SAFE" for r in files)
        susp = sum(r["status"] == "SUSPICIOUS" for r in files)
        quar = sum(r["decision"] == "QUARANTINE" for r in files)
        return render_template_string(PAGE, files=files, safe=safe,
                                      susp=susp, quar=quar)

    return app
```

Description: saves the JSON report, sends the “USB Security Scan Report Generated” e-mail with the report attached, and renders the dashboard summary cards (Safe Files, Suspicious, Quarantine) and results table.

4.1.4 Code Execution Flow Chart

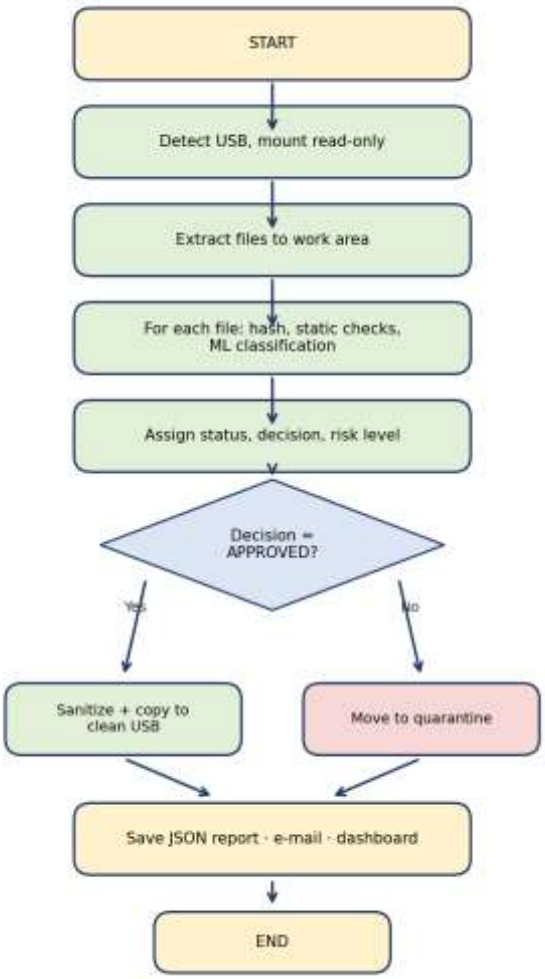


Figure 4.1.4: Code Execution Flow Chart

The flow chart traces one inspection: files are extracted, scanned and classified, each receives a decision, approved files are sanitized and copied to the clean USB, threats are quarantined, and the report is saved, e-mailed and shown on the dashboard.

4.2 TEST CASE STUDY

4.2.1 Test Case 1

Test Objective: Verify the application starts and loads existing reports.

Test Steps: Activate the virtual environment, go to the app folder and run python main.py.

Expected Result: The program lists stored reports and reads the latest.

Actual Result: All reports (scan_report_20260929 … scan_report_20261011_195046.json) were listed and the latest was read; 7 files in the report.

Status: Pass

4.2.2 Test Case 2

Test Objective: Verify detection and classification of files on an inserted USB.

Test Steps: Insert a test USB containing normal text/CSV files and script/executable files; start the scan.

Expected Result: Normal files are APPROVED (LOW risk); scripts and executables are QUARANTINE.

Actual Result: normal_test_11.txt, normal_data_11.csv, WPSettings_10.dat and IndexerVolumeGuid_10 were APPROVED (LOW); test_3.bat, test_2.bat and test.exe were QUARANTINE (MEDIUM).

Status: Pass

4.2.3 Test Case 3

Test Objective: Verify report generation and e-mail notification.

Test Steps: Complete a scan and open the mailbox.

Expected Result: An e-mail with the scan report attached is received.

Actual Result: The e-mail "USB Security Scan Report Generated" arrived with the report path, timestamp and scan_report_…json attachment.

Status: Pass

4.2.4 Test Case 4

Test Objective: Verify the web dashboard.

Test Steps: Open Chromium and browse to the kiosk's local address on port 5000.

Expected Result: The dashboard displays totals and per-file results.

Actual Result: The dashboard showed 4 Safe Files, 0 Suspicious, 3 Quarantine and the scan results table.

Status: Pass

4.3 TEST SUMMARY

Table 4.3.1 summarises the four test cases of Section 4.2. All four tests produced the expected result on the prototype.

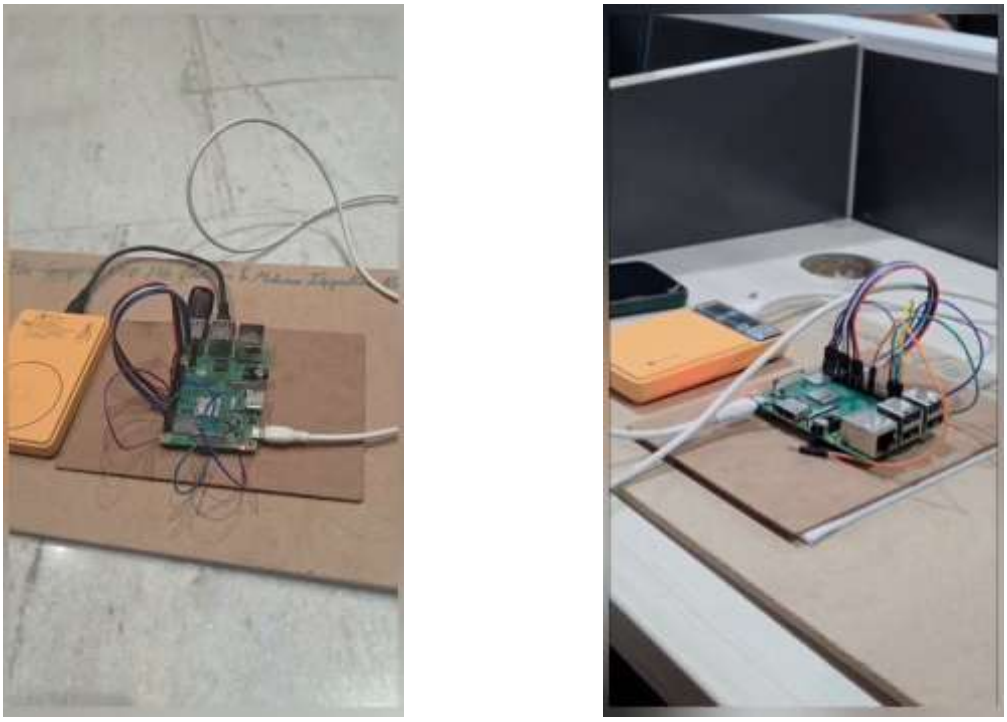
Test	Objective	Expected result met	Status
TC1	Application starts and loads existing reports	Yes	Pass
TC2	Detection and classification of files on the USB	Yes	Pass
TC3	Report generation and e-mail notification	Yes	Pass
TC4	Web dashboard shows totals and per-file results	Yes	Pass

Table 4.3.1: Summary of Test Cases

CHAPTER - 5
RESULTS

5.1 RESULTING IMAGES

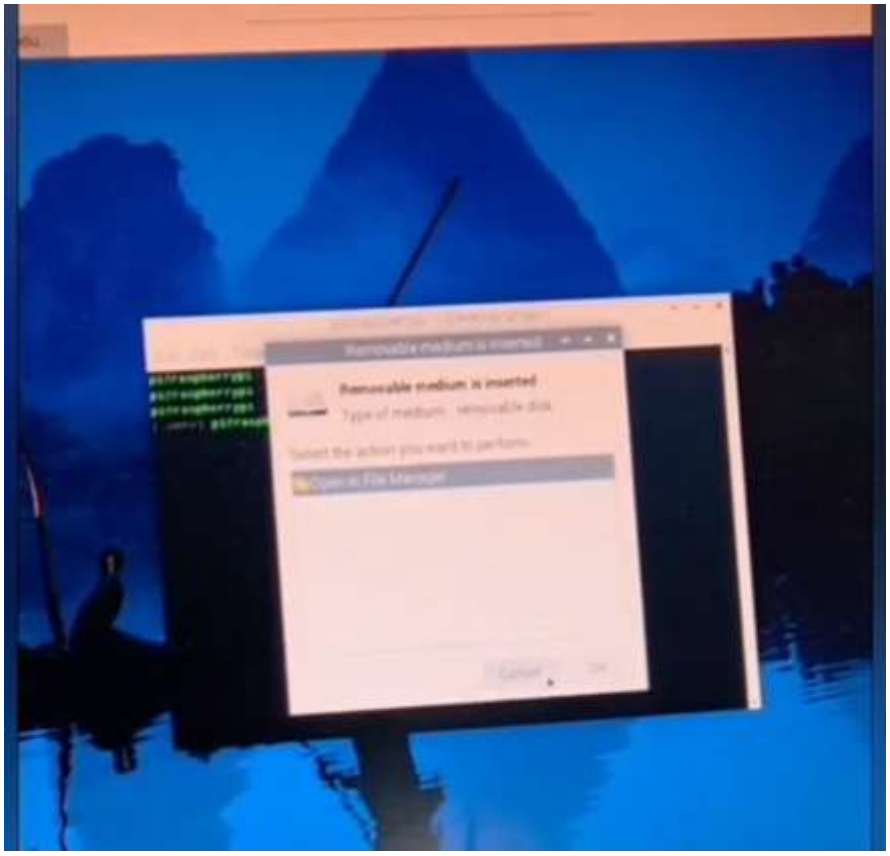
5.1.1 Screenshot 1



Screenshot 5.1.1: Kiosk Hardware – front and side views of the Raspberry Pi with GPIO wiring, USB drive and portable storage

The prototype is mounted on a base board labelled with the project name. The Raspberry Pi carries jumper-wire connections, the untrusted USB drive in the input port, a portable storage drive and the power cable.

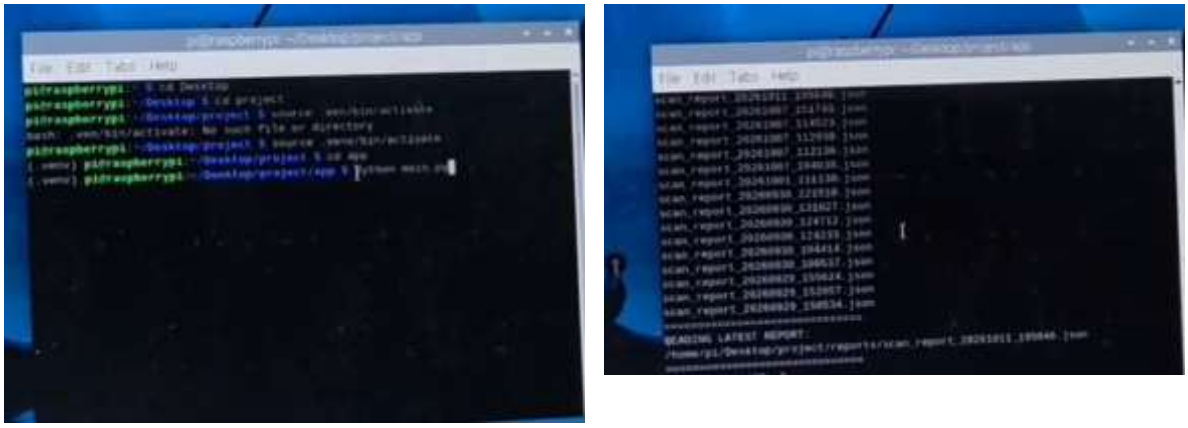
5.1.2 Screenshot 2



Screenshot 5.1.2: Removable Medium Detection – “Removable medium is inserted” prompt on the kiosk

When the USB is inserted, the Raspberry Pi OS detects the removable medium and prompts for the action to perform.

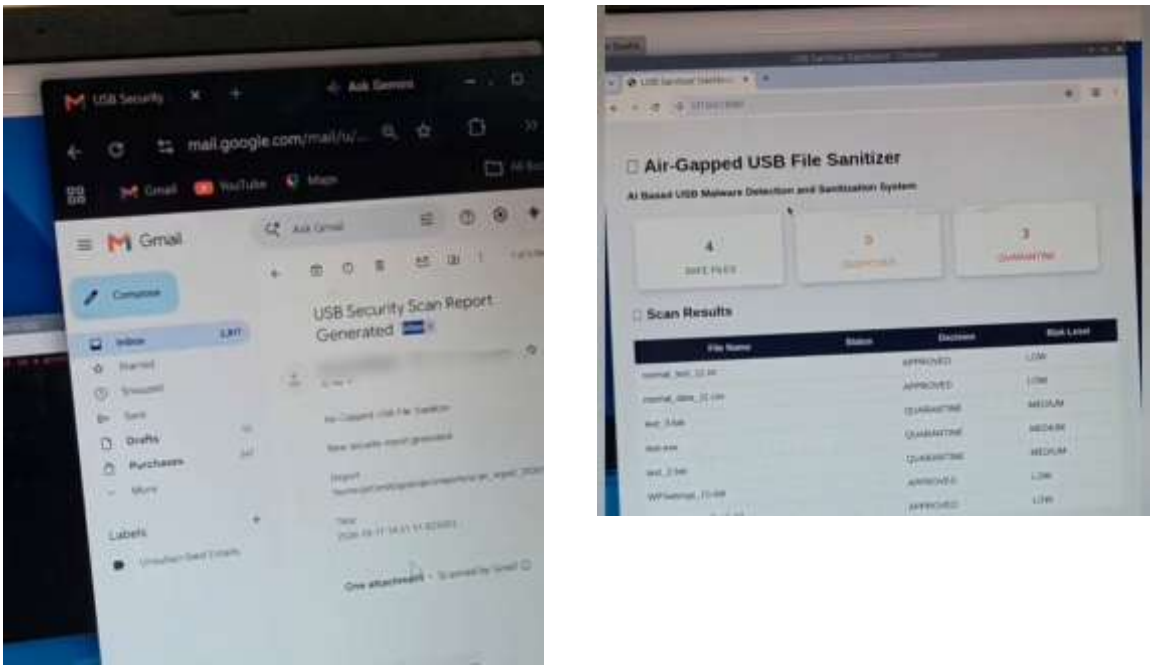
5.1.3 Screenshot 3



Screenshot 5.1.3: Application Start and Report Listing (python main.py)

The application is started from the virtual environment; it lists all stored scan_report_*.json files, reads the latest report (7 files) and starts the dashboard.

5.1.4 Screenshot 4



Screenshot 5.1.4: E-mail Report (left) and USB Sanitizer Dashboard (right)

The e-mail "USB Security Scan Report Generated" carries the report path, time and JSON attachment. The dashboard titled "Air-Gapped USB File Sanitizer – AI Based USB Malware Detection and Sanitization System" shows 4 Safe Files, 0 Suspicious, 3 Quarantine and a results table.

5.2 RESULTING TABLES

5.2.1 Table 1

File Name	Decision	Risk Level
normal_test_11.txt	APPROVED	LOW
normal_data_11.csv	APPROVED	LOW
test_3.bat	QUARANTINE	MEDIUM
test.exe	QUARANTINE	MEDIUM
test_2.bat	QUARANTINE	MEDIUM
WPSettings_10.dat	APPROVED	LOW
IndexerVolumeGuid_10	APPROVED	LOW

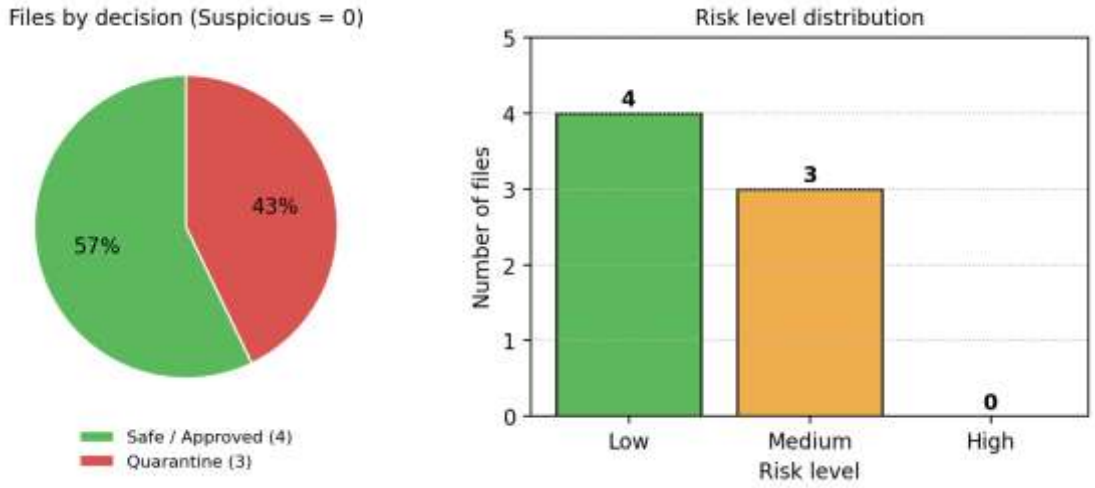
Table 5.2.1: Per-file Results of the Demonstration Scan

5.2.2 Table 2

Metric	Value	Share
Total files	7	100%
Safe / Approved	4	57%
Suspicious	0	0%
Quarantined	3	43%
Status	Threats detected	–

Table 5.2.2: Summary of the Demonstration Scan

5.3 RESULTING GRAPHS



Graph 5.3.1: Files by Decision and Risk Level Distribution

The graphs show that all script and executable files were isolated in quarantine, while ordinary data and system files were approved for transfer.

5.4 RESULTS & ANALYSIS

5.4.1 Time Complexity

For n files, hashing, scanning and classification each take time proportional to file size, so overall scanning is O(n·s) for average file size s. Rule checks per file take O(1), and building the report is O(n). These are theoretical estimates; measured scan times can be added after further testing.

5.4.2 Space Complexity

Storage is dominated by the extracted working copy and the quarantine area, O(n·s) in the worst case. Each JSON report is small, with one record per file.

5.4.3 Results Summary

The prototype demonstrates the full workflow on real hardware: insertion, isolation, scanning, classification, quarantine, report generation, e-mail notification and dashboard display. Of 7

files, 4 were approved and 3 were quarantined, and every scan is preserved as a timestamped JSON report for auditing.

5.4.4 Discussion & Limitations

The demonstration scan confirms that the complete workflow runs on the prototype hardware and that the decision policy behaves as designed: ordinary text, CSV and system files were approved, while the two batch scripts and the executable were quarantined. The scan contained only seven files, so the results show that the pipeline works end to end; they are not a statistical measure of detection accuracy.

Several limitations should be noted. The demonstration did not include samples assigned a HIGH risk level, so the behaviour for that class has not yet been demonstrated on the prototype. Scan times were not measured, and the complexity figures above are theoretical. The ML classifier relies on file features, so it can produce false positives on legitimate scripts and may miss threats that are well disguised. Finally, the dashboard currently runs in debug mode without authentication, which is suitable for a prototype but must be changed before production use.

- **Larger test set:** Phase-2 should test a wider range of file types, sizes and known malware samples in a controlled environment.
- **Measured performance:** scan time and memory should be measured against the number and size of files.
- **Accuracy metrics:** false positive and false negative rates should be recorded for the ML classifier.
- **Hardening:** authentication, a production web server and tamper protection should be added.

5.5 COMPARISON WITH EXISTING APPROACHES

Table 5.5.1 compares the proposed kiosk with the approaches reviewed in Section 2.1. The comparison is qualitative and is based on the design of each approach and on the limitations identified in Chapter 2; it is not the result of a benchmark.

Feature	Direct connection	Software scanning PC	Commercial kiosk	Proposed kiosk
Physical isolation of media	No	Partial	Yes	Yes
Dedicated inspection device	No	Partial	Yes	Yes
AI/ML classification	Varies	Varies	Varies	Yes
Quarantine of threats	Varies	Varies	Yes	Yes
Clean output path	No	No	Yes	Yes
Timestamped audit reports	No	Varies	Yes	Yes
Low cost and open source	Yes	Yes	No	Yes

Table 5.5.1: Qualitative Comparison of USB Inspection Approaches

The main advantage of the proposed kiosk is that it provides the isolation, quarantine and clean-output behaviour of a commercial station on inexpensive, open hardware, while remaining modular enough for scanners and models to be replaced as threats evolve.

CHAPTER - 6
CONCLUSION & FUTURE SCOPE

6.1 CONCLUSION

The Air-Gapped USB File Sanitizer & Malware Inspection Kiosk acts as a security checkpoint between an untrusted USB and a trusted computer. It integrates Cyber Security, AI/ML, IoT, Embedded Hardware, sandbox analysis and controlled file sanitization on a low-cost platform. The core objective was achieved: inspect first, isolate threats, and transfer only approved files.

Inspect first. Isolate threats. Transfer only what is safe.

6.2 FUTURE SCOPE

- Advanced deep-learning malware detection.
- Improved zero-day threat detection.
- More powerful sandboxing.
- Automated file/document reconstruction.
- Hardware tamper detection.
- Secure centralized reporting.
- Controlled cloud-based threat intelligence.
- Multi-kiosk security management.
- Production hardening of the web application (disabling debug mode, adding authentication to the dashboard).

6.3 LEARNING OUTCOMES

Working on Phase-1 gave the team practical experience in several areas that are central to cyber security engineering.

- **Malware handling concepts:** hashing, static and dynamic analysis, sandboxing and quarantine.
- **Embedded and IoT development:** Raspberry Pi setup, Linux administration, USB mounting and GPIO wiring.
- **Applied AI/ML:** using file features to classify files and assign risk levels.
- **Software engineering:** modular Python design, Flask web development, JSON reporting and e-mail automation.
- **Security engineering:** trust boundaries, fail-safe decisions and audit logging.
- **Teamwork and documentation:** planning, testing and reporting a multi-module project.

REFERENCES

1. K. Nohl and J. Lell, “BadUSB – On accessories that turn evil,” Black Hat USA, 2014.
2. M. Sikorski and A. Honig, Practical Malware Analysis, No Starch Press, 2012.
3. NIST, Guide to Malware Incident Prevention and Handling for Desktops and Laptops, SP 800-83 Rev. 1, 2013.
4. NIST, Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5, 2020.
5. Raspberry Pi Ltd., Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/
6. Flask Documentation. https://flask.palletsprojects.com/
7. Python Software Foundation, Python 3 Documentation. https://docs.python.orgw/3/